Université de Lille

FACULTÉ DES SCIENCES ET TECHNOLOGIES
Département Informatique

Projet de Qualité Logicielle

Réalisé par :
- Ali EL HASSNAOUI.
- Roderick Darell Tchinde Singhe

Encadré par : Mme Clotilde Toullec.

Sommaire

- Projet de Qualité Logicielle
  - Sommaire
  - 1. Présentation globale du projet
    - 1.1 Objectifs
    - 1.2 Utilité du projet
    - 1.3 Description du projet
  - 2. Historique du logiciel
    - 2.1 Analyse du dépôt Git
  - 3. Architecture logicielle
    - 3.1 Utilisation de bibliothèques extérieures
    - 3.2 Organisation en paquetages
    - 3.3 Répartition des classes dans les paquetages
    - 3.4 Organisations des classes
  - 4. Analyse approfondie
    - 4.1 Tests
    - 4.2 Commentaires
    - 4.3 Dépréciation
    - 4.4 Duplication de code
    - 4.5 God Classes
    - 4.6 Proportion des lignes de code
    - 4.7 Analyse des méthodes
    - 4.8 Complixité
      - Complexité cyclomatique
      - Complexité cognitive
  - 5. Nettoyage de code et code smells
    - 5.1 Règles de nommage
    - 5.2 Nombres magiques
    - 5.3 Code mort
  - 6. Synthèse critique

1. Présentation globale du projet

1.1 Objectifs

L'objectif principal est de réaliser une analyse approfondi du projet **Red5-Server**. Plus spécifiquement :

- **Comprendre la structure** du logiciel, ainsi que la répartition architecturale de ses composants ("packages", classes et bibliothèques).
- **Évaluer la qualité** intrinsèque du code via différentes métriques (tests, couverture, commentaires etc).
- **Analyser l'état et l'historique** du projet par l'étude de son référentiel Git (équité des contributions, rythme de développement etc).

Il s'agit donc d'identifier les points de force de l'implémentation et de cerner les défis liés à la maintenabilité ou au nettoyage de dettes techniques qui pourraient s'être accumulées tout au long du cycle de vie du projet.

1.2 Utilité du projet

- **Utilité et fonctionnalités :** Le projet Red5 Server est un serveur média performant écrit en Java. Il vise à offrir une solution open-source de diffusion de flux en continu (streaming) audio/vidéo. Il permet également d'enregistrer les flux clients (live stream publishing) et propose un support intégré pour la collaboration temps-réel.
- **Lancement :** Le lancement et le build du serveur s'appuient sur Maven. On compile par exemple le projet avec `mvn -Dmaven.test.skip=true install`.
- **Résultat du projet :** En sortie, la compilation génère un ensemble de fichiers d'archives packagés au format JAR (Core, Server, IO).

1.3 Description du projet

- **Documentation (`README.md`) :** Un fichier `README.md` exhaustif est bien présent. Il est maintenu à jour et fait un excellent résumé du statut actuel du système, de l'historique et des pré-requis.

- **Contenu de la documentation :** Si le `README.md` est extrêmement clair sur l'organisation des sous-modules (`common` , `io` , `server` , etc.), il redirige une grande partie de la question "comment déployer en profondeur" vers la base documentaire générale (wiki ou sources). Toutefois, les instructions de lancement Maven sont très précises et claires pour le développeur, ce qui suffit à l'usage premier de la compilation.

- **Pertinence :** La documentation est très pertinente : on n'y manque véritablement de rien pour lancer le mode développeur ou builder le produit.

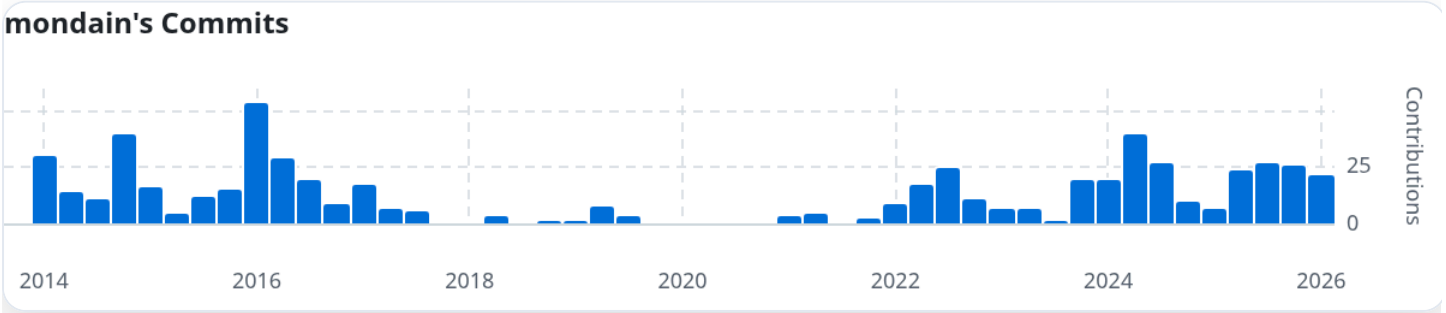
---

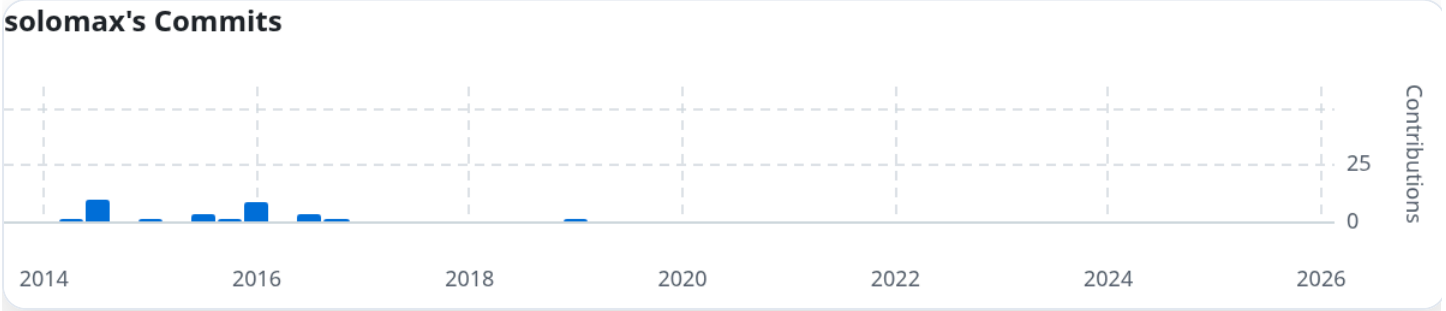
2. Historique du logiciel

2.1 Analyse du dépôt Git

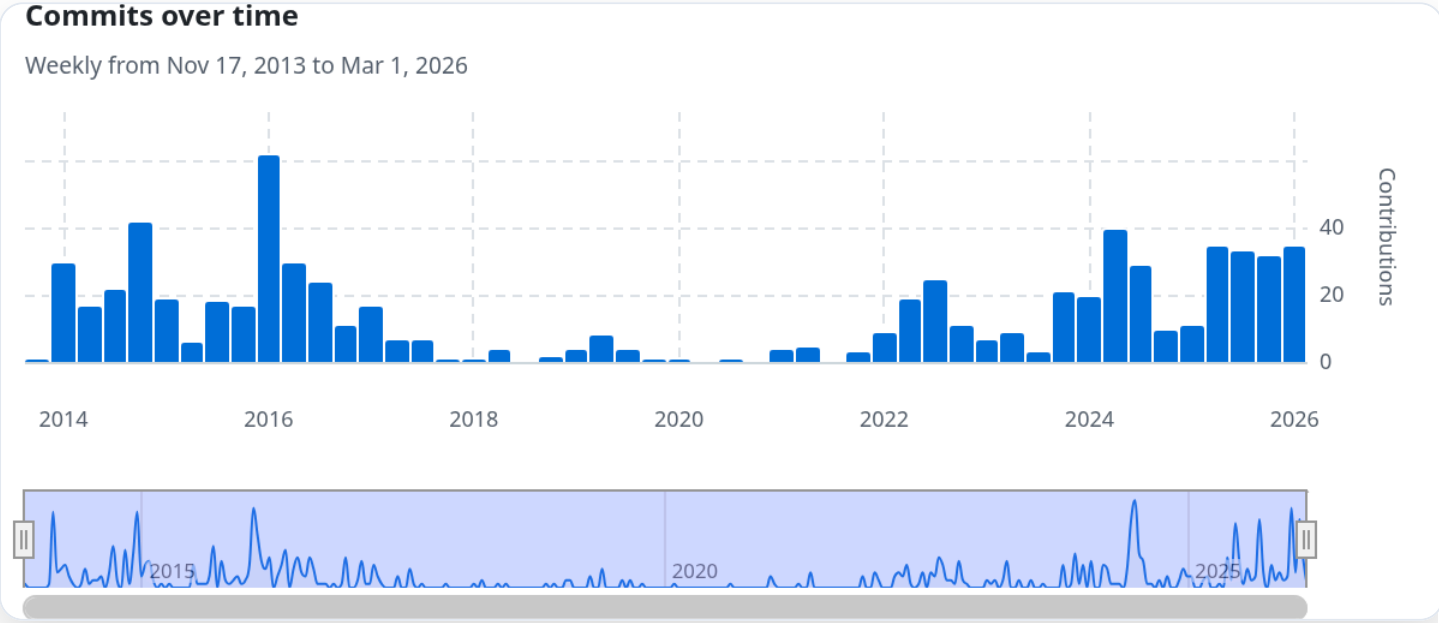
L'historique Git (analysable via les logs) illustre la longue maturité du projet.

- **Composition de l'équipe :** La composition affiche un nombre impressionnant de contributeurs pour ce projet. Les noms majeurs comme "Paul Gregoire" comptabilisent de très loin l'immense majorité des commits, suivi par "Mondain" et d'autres profils. On observe donc que la contribution n'est pas équilibrée; elle gravite autour de deux développeurs.



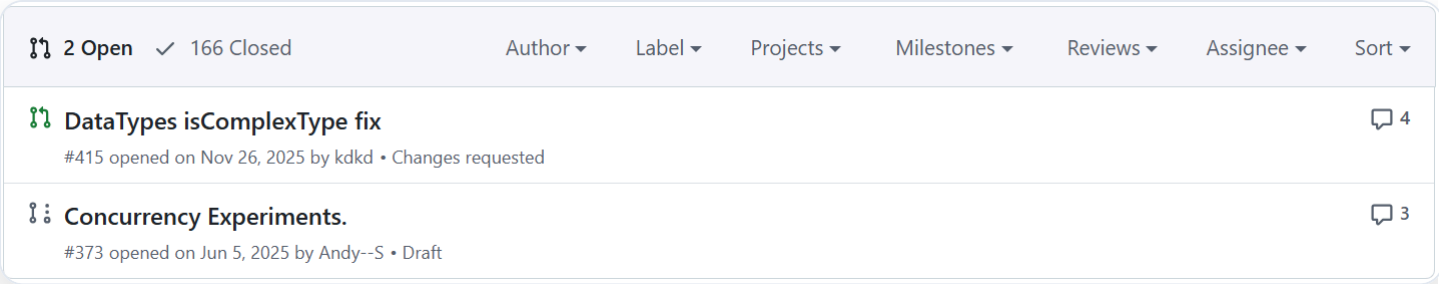


- **Activité sur le projet :** L'activité de ce serveur est globalement régulière entre 2014 et 2026, à l'exception de la période 2018-2022 durant laquelle elle est un peu plus faible, car le marché et les développeurs "core" se retournaient vers de nouveau protocoles comme le `WebRTC` (comercial) sur Red5 Pro.

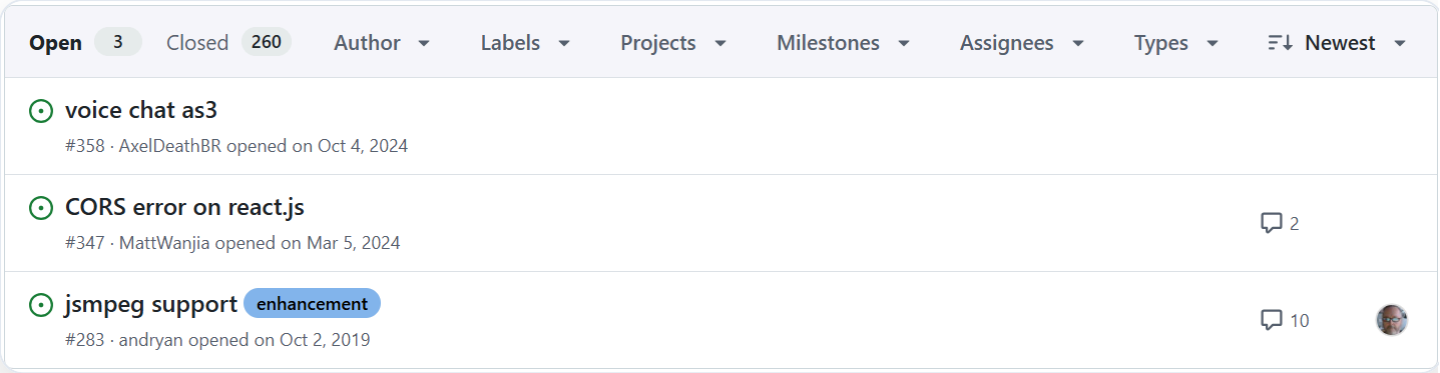


- **Branches :** Le projet a totalisé **10** branches au cours de son existence dont deux actuellement encoreactive(hors branches principale) pour un total de **4** contributeurs impliqués dans ces branches est autour d'une branche principale et exploite ponctuellement des branches secondaires. Ces dernières ciblent l'ajout de correctifs de sécurité ou l'expérimentation de fonctionalités.

- **Pull requests :** la composition affiche **166** pull requests sont totalement résolus et **2** sont signalés. On remarque un faible taux de pull request de MONDAIN par rapport a son nombre de commits ce qui sous -entend qu'il a eu a effectuer direcctement des push. ces indicateurs soulignent que le mécanisme de Pull Request est fortement actif pour la sécurité des paquets et la validation des contributions par les développeurs.



- **Issues :** La section affiche **260** issues totalement résolues (fermées) et seulement **3** sont signalées comme ouvertes (principalement des améliorations externes). On remarque un très faible taux d'issues non traitées par rapport au volume total, ce qui sous-entend que l'équipe s'est attelée à corriger rapidement et régulièrement les remontées. Ces indicateurs soulignent que le suivi des incidents est fortement actif pour garantir la stabilité du serveur et la validation des correctifs par les développeurs.



3. Architecture logicielle

3.1 Utilisation de bibliothèques extérieures

- Le serveur fait un usage massif et justifié de bibliothèques Java très importantes. Le fichier `pom.xml` global importe des technologies dominantes : `Spring Framework` , `Apache Mina` , `Tomact` , `SLF4J` et `Logback` .
- **Utilisation :** Les dépendances clés (telles que `Spring` , `Tomcat` et `SLF4J`) sont absolument indispensables au projet pour éviter de redévelopper une gestion complexe des cycles de vie asynchrones (sockets RTMP)

3.2 Organisation en paquetages

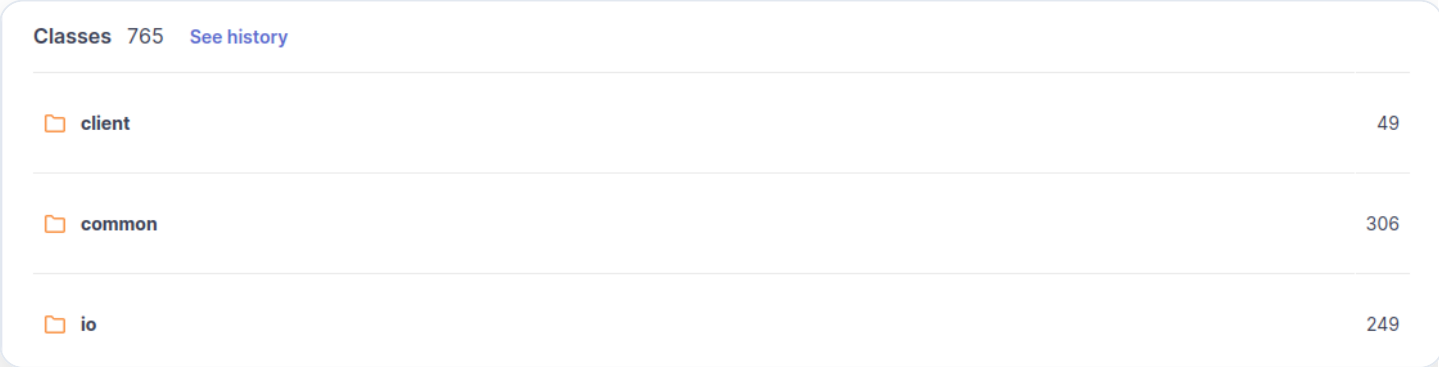
- **Nombre et distribution :** Un scan global du projet recense **125** paquetages définis uniquement pour le code source de production.

- **Hiérarchie :** L'organisation par couche est rigoureusement structurée. L'arborescence respecte une séparation fine : `org.red5.server`, `org.red5.io`, `org.red5.client`. Les tests ont hérité de ces mêmes noms de paquets (dans `src/test/java`).

- **Nommage (Design Patterns) :** Les noms des paquetages (`org.red5.server.plugin`, `messaging`, `stream`) traduisent explicitement l'architecture : système modulaire d'extension `plugins`, architecture de propagation de message `messaging` etc.

3.3 Répartition des classes dans les paquetages

- Le projet regroupe une moyenne d'un peu plus de **766 fichiers Java**. En moyenne, chaque paquetage comporter 765/125 ~= **6 classes**.



3.4 Organisations des classes

- **Hiérarchie (DIT et NOC) :** La hiérarchie est peu profonde, utilise l'héritage de manière contrôlée et favorise la réutilisation du code. L'arbre d'héritage est mesuré mais robuste : de nombreux design patterns imposent d'implémenter des interfaces complexes (`IConnection`, `IStream`).

- **Stabilité et Cohésion :** La cohésion de paquets comme `org.red5.io.flv` est très forte. En revanche, la cohésion de paquets plus centraux agissant en tant que médiateurs semble s'éparpiller légèrement de par leur rôle "orchestrateur" global.

4. Analyse approfondie

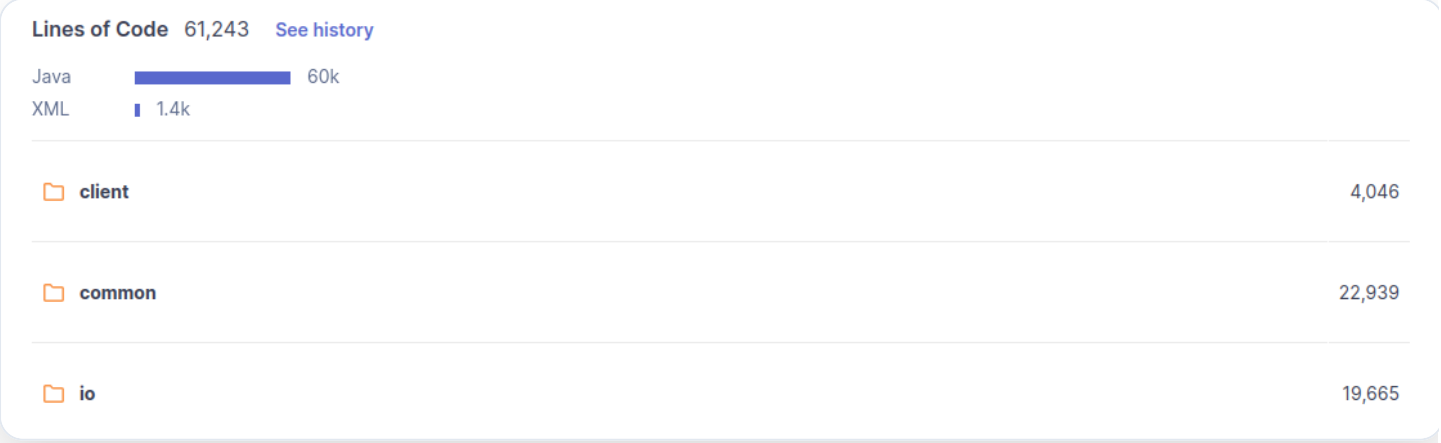
4.1 Tests

- **Périmètre des tests :** Le code dispose de **66** classes Java pour effectuer les tests, ce qui est assez mince en proportion des **766** classes métier (c-à-d : **1/11**).
- **Cible et assertions :** Ces tests unitaires permettent de s'assurer du fonctionnement logique (I/O etc).
- **Couverture :** Concernant la couverture, une valeur de **0 %** a été observée dans le rapport disponible. cela est du à une mauvaise configuration du pom qui empeche notre jacoco d'effectuer une analyse correcte sous sonar. Un autre logiciel de mesure nous revèle un pourcentage de couverture de test d'environs 56% ce qui est très faible.

Element	Class, %	Method, %	Line, %	Branch, %
all	26% (188/703)	18% (1029/5664)	15% (4421/29333)	9% (1422/14557)
org.red5	26% (188/702)	18% (1029/5663)	15% (4421/29332)	9% (1422/14557)
Version	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

4.2 Commentaires

- Lignes de code **LOC 61 243**



- Lignes de commentaires : **LC 25 137**

Comment Lines 25,137 See history	
client	1,163
common	11,574
io	7,544

- Pourcentage : **41.04%** (soit environ **41** lignes commentées pour **100** lignes de code exécutable).

- **Diagnostic :** La documentation est excellente (**41.04%** des lignes sont commentées), ce qui indique un effort de documentation significatif.

4.3 Dépréciation

- Le passage techniques à de nouvelles versions (beaucoup plus performantes et plus fluides) de **Spring** ou d'**Apache Mina** ont déprécié certaines méthodes comme `handleConnection()`. Des méthodes marquées `@Deprecated` survivent au profit de versions plus performantes évitant souvent des appels bloquants.

4.4 Duplication de code

- La présence de classes "God" (voir ci-dessous) suggère potentiellement une duplication de logique interne (copier-coller de blocs de code)

Duplicated Lines 3,491 See history		
	Duplicated Lines	Duplicated Lines (%)
client	394	5.9%
common	1,449	2.9%
io	1,166	3.1%

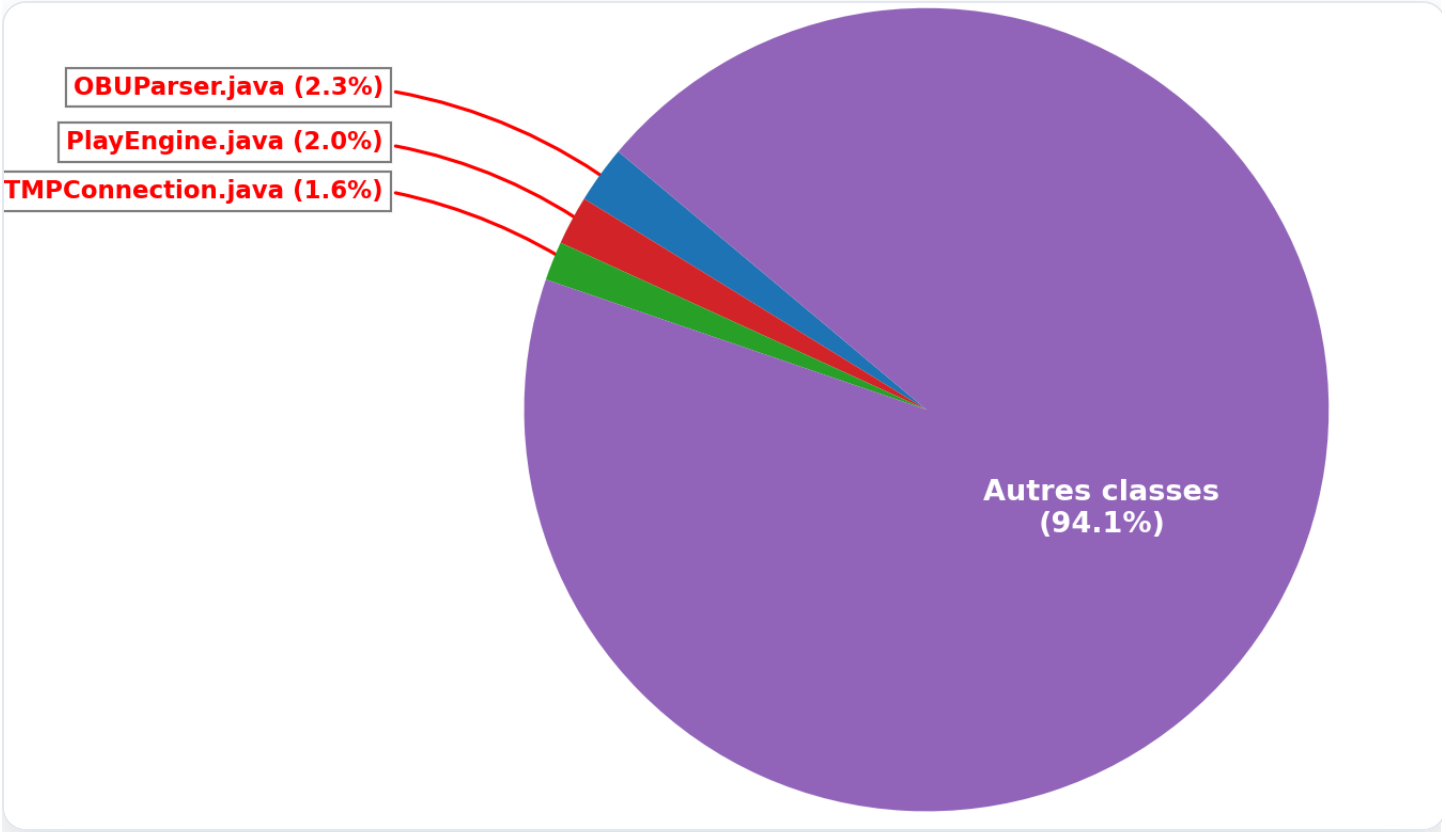
- **Densité** : représente le pourcentage **moyenne** de lignes de code qui sont dupliquées dans notre cas **2.9%** sur un total de **61 243** lignes de code (environ **1776** lignes dupliquées). Le module server, qui gère le cœur architectural complexe (les connexions, etc.), ne présente que **1.7%** de duplication. Cela confirme qu'il est extrêmement bien écrit et structuré.

4.5 God Classes

Notre analyse remonte des "God Classes" évidentes :

- `PlayEngine.java` : Avec environ **1449 lignes** et plus de **64 méthodes**. Le nom "Engine" est souvent synonyme de "God Class". Elle manipule l'intégralité du temps, des états, et du cycle de vue "Play".
- `RTMPConnection.java` : **1152 lignes**, **125 méthodes**. Cette classe encapsule tout ce qui a trait aux caractéristiques d'une connexion réseau en streaming sur le protocole principal RTMP. Ses variables d'instance et sa complexité sont gigantesques.
- `OBUParser.java` : **1696 lignes**. Bien qu'immense, c'est généralement toléré car auto-généré ou contenant de gros mapping en format parseur.

4.6 Proportion des lignes de code



- Pertinence des pourcentages : Bien que ces trois classes ne représentent chacune que **1,6 %** à **2,3 %** du volume global, elles demeurent des God Classes avérées. Dans une architecture massive et modulaire comme Red5 (plus de **62 000** lignes de code et **766** dépendances/fichiers class), un fichier unique de **1 500** lignes est gigantesque par rapport à la taille moyenne d'une classe standard.

4.7 Analyse des méthodes

Les méthodes les plus complexes se situent logiquement dans les zones liées au décodage de protocole, à la lecture de flux et aux mécanismes réseau.

La classe `RTMPProtocolDecoder.java`, par exemple, présente naturellement une forte complexité cyclomatique en raison de la logique de dispatch, des branchements et des traitements dépendants des octets lus.

Dans ce type de système, une complexité élevée n'est pas toujours évitable. En revanche, elle devient problématique lorsqu'elle se combine avec :

- un grand nombre de responsabilités dans une même méthode ;
- des effets de bord sur l'état d'objets partagés ;
- une testabilité faible ;
- une lisibilité dégradée.

C'est précisément ce risque que l'on observe sur certaines parties centrales de Red5-Server.

4.8 Complixité

Complexité cyclomatique

D'après l'analyse réalisée avec Sonar, les fichiers présentant la **plus forte complexité cyclomatique** sont :

- **OBUParser.java** avec une complexité de **439** ;
- **PlayEngine.java** avec **361** ;
- **RTMPConnection.java** avec **300**.

Le cas de **OBUParser.java** peut être partiellement expliqué par la nature même de la classe. Le terme *Parser* indique qu'elle est responsable de l'analyse d'un format ou d'un protocole. Ce type de composant doit souvent gérer de nombreux cas, structures conditionnelles et traitements différents. Une complexité élevée est donc relativement attendue dans ce type de classe.

En revanche, **PlayEngine** et **RTMPConnection** ont également été identifiées précédemment comme des **God Classes potentielles**. Leur complexité importante peut donc être liée au fait qu'elles concentrent plusieurs responsabilités dans un même fichier. Une décomposition de ces classes en composants plus spécialisés pourrait permettre de réduire cette complexité lors d'une phase d'amélioration ultérieure.

Un autre point important est que **les trois fichiers les plus complexes représentent à eux seuls environ 8 % de la complexité cyclomatique totale**, calculée sur environ **660 fichiers analysés**. Cela montre que la complexité du projet est **fortement concentrée dans un nombre limité de classes centrales**.

Complexité cognitive

io/src/main/java/org/red5/io/obu/OBUParser.java	727
common/src/main/java/org/red5/server/stream/PlayEngine.java	716
io/src/main/java/org/red5/io/mp4/impl/MP4Reader.java	437

Concernant la **complexité cognitive**, les fichiers les plus concernés sont :

- **OBUParser.java** ;
- **PlayEngine.java** ;
- **MP4Reader.java**.

Dans cette métrique également, **PlayEngine** apparaît comme un point particulièrement sensible. Sa valeur atteint environ **716**, tandis que le troisième fichier présente une valeur d'environ **437**, soit presque la moitié de celle de PlayEngine.

Cette observation confirme que certaines classes jouent un rôle central dans le système et concentrent une grande partie de la logique métier. Comme pour la complexité cyclomatique, une **répartition plus équilibrée des responsabilités** dans plusieurs classes pourrait améliorer la lisibilité et la maintenabilité du code.

De manière générale, ces résultats montrent que **la complexité globale du projet n'est pas uniformément répartie**, mais plutôt concentrée dans quelques composants clés du système. Ce phénomène est fréquent dans les logiciels de grande taille et constitue un indicateur utile pour identifier les zones qui pourraient être ciblées lors d'une future phase d'amélioration.

5. Nettoyage de code et code smells

5.1 Règles de nommage

Dans l'ensemble, le projet respecte plutôt bien les conventions Java classiques. Les noms liés au domaine — `stream`, `packet`, `decoder`, `session`, `message`, `connection` — sont globalement parlants et cohérents avec le contexte applicatif.

Cependant, les outils d'analyse ont relevé plusieurs anomalies.

- **Constantes et conventions :**

```
private static final boolean isDebug = log.isDebugEnabled();
```

Rename this constant name to match the regular expression '^[A-Z][A-Z0-9]*(_[A-Z0-9]+)*$'. 2 years ago L103
Why is this an issue?
Code Smell Critical Open Not assigned 2min effort Comment convention

Nous avons relevé environ **106 issues** de type *"Constant names should comply with a naming convention"*. Toutes ne semblent pas également graves, mais elles montrent qu'une partie des constantes n'est pas nommée de façon homogène.

- **Problèmes d'héritage (shadowing de champs) :**

Un autre problème identifié concerne les cas où une classe fille redéclare un champ déjà présent dans sa classe parente. Ce phénomène est appelé **shadowing**.

- Environ **34 issues** de type *"Child class fields should not shadow parent class fields"* ont été relevées.

Ce problème peut rendre le code plus difficile à comprendre, car il devient moins clair de savoir **quel champ est réellement utilisé** (celui de la classe parente ou celui de la classe fille). Cela peut aussi provoquer des erreurs lors de la maintenance du code.

```
ublic class RTMPSClient extends RTMPClient {

    private static String[] cipherSuites;

    // I/O handler
    private RTMPSClientIoHandler ioHandler;
```

'ioHandler" is the name of a field in "RTMPClient". Why is this an issue? 8 months ago L48
Code Smell Blocker Open Not assigned 5min effort Comment confusing

- **Ambiguïtés et collisions de noms :**

D'autres remarques concernent :

- des appels ambigus à des méthodes de super-classe depuis des classes internes ;
- des noms de classes qui entrent en collision avec ceux d'interfaces ou de super-classes ;
- certaines déclarations peu lisibles où le type final ou l'intention réelle devient difficile à saisir rapidement.

Dans l'ensemble, le nommage reste donc **majoritairement cohérent**, mais il existe des zones localisées où la qualité baisse nettement.

5.2 Nombres magiques

Le projet contient un grand nombre de signalements liés aux nombres magiques. Une valeur brute comme **1878 issues** a été observée, mais elle doit être interprétée avec précaution, car l'outil inclut aussi des cas peu pertinents, comme certains accès à des indices de tableaux.

Les nombres les plus intéressants sont ceux qui portent une signification métier ou protocolaire.

Dans cette catégorie, l'outil d'analyse détecte un grand nombre de **nombres magiques** (valeurs numériques utilisées directement dans le code sans constante explicite). Ces valeurs peuvent correspondre à des paramètres techniques, à des tailles de buffers ou encore à des valeurs propres à certains protocoles.

La figure associée montre plusieurs exemples détectés par l'outil d'analyse. Certaines valeurs ont une signification claire liée au protocole ou au fonctionnement interne du système. Dans ce cas, elles ne sont pas nécessairement incorrectes, mais elles pourraient être rendues plus explicites à l'aide de constantes nommées.

Dans le cadre de ce rapport, l'objectif est uniquement **d'identifier et d'expliquer ces occurrences**, sans modifier le code du projet. L'image associée présente quelques exemples représentatifs de ces valeurs numériques. Par exemple, certaines valeurs correspondent au **port standard du protocole RTMP utilisé pour le streaming audio/vidéo**, d'autres à la **longueur du digest utilisé lors du handshake RTMP**, et d'autres encore à la **taille d'un buffer temporaire utilisé pour traiter les headers ou les paquets du protocole**. Ces nombres ne sont donc pas forcément incorrects : ils ont souvent une **signification technique précise** dans le fonctionnement du système. L'analyse consiste donc simplement à expliquer ce que ces valeurs représentent et pourquoi elles apparaissent dans le code. Dans une phase ultérieure du projet, elles pourraient être remplacées par des constantes explicites afin d'améliorer la lisibilité et la maintenabilité du code.

```
    Output.putString(result, header.getName());
    result.put(header.getMustUnderstand() ? (byte) 0x01 : (byte) 0x00);

    IoBuffer tmp = IoBuffer.allocate(1024);
```

```
     *            Method parameters
     * @return Byte buffer with data to perform remoting call
     */
    private IoBuffer encodeInvoke(String method, Object[] params) {
        log.debug("RemotingClient encodeInvoke - method: {} params: {}", method, params);
        IoBuffer result = IoBuffer.allocate(1024);
```

Assign this magic number 1024 to a well-named constant, and use the constant instead. 4 years ago L101
Why is this an issue?
Code Smell Major Open Not assigned 5min effort Comment brain-overload

```
        result.setAutoExpand(true);
        //force version 3
        result.putShort((short) 3);
```

Assign this magic number 3 to a well-named constant, and use the constant instead. 4 years ago L104
Why is this an issue?
Code Smell Major Open Not assigned 5min effort Comment brain-overload

```
        // Headers
        Collection<RemotingHeader> hdr = headers.values();
        result.putShort((short) hdr.size());
        for (RemotingHeader header : hdr) {
            Output.putString(result, header.getName());
            result.put(header.getMustUnderstand() ? (byte) 0x01 : (byte) 0x00);

            IoBuffer tmp = IoBuffer.allocate(1024);
```

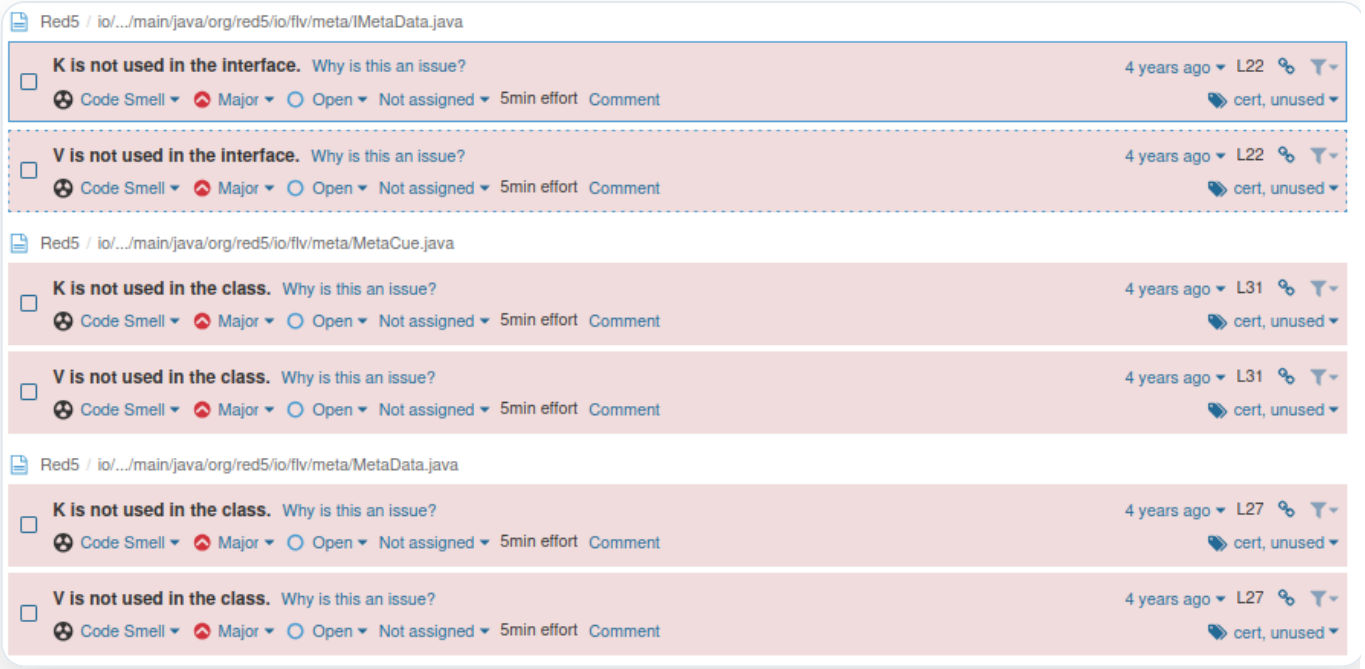
Assign this magic number 1024 to a well-named constant, and use the constant instead. 4 years ago L112

Bilan : tous les nombres magiques ne sont pas de mauvaises pratiques au même degré. Dans Red5-Server, beaucoup relèvent du domaine protocolaire. Le vrai enjeu est donc moins leur suppression que leur **mise en explicitation**.

5.3 Code mort

On remarque la présence de code mort sous plusieurs formes dans ce projet. On peut notamment citer :

- **Types inutilisés :**



Un autre signal intéressant concerne les *unused type parameters*. Deux interprétations sont possibles :

- soit il s'agit d'un résidu de conception, devenu inutile avec l'évolution du code ;
- soit il s'agit d'une généralisation anticipée, pensée pour un usage futur jamais concrétisé.

Dans les deux cas, cela mérite une relecture, car ce type d'abstraction inutile complique la compréhension sans apporter de bénéfice immédiat.

- Paramètres de méthodes inutilisés :

Red5 / io/.../main/java/org/red5/io/mp4/impl/MP4Reader.java

Remove this unused method parameter "scale". Why is this an issue? 2 years ago L584
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "scale". Why is this an issue? 2 years ago L683
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/object/Serializer.java

Remove these unused method parameters. Why is this an issue? 2 years ago L74
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/src/main/java/org/red5/io/obu/OBUParser.java

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L917
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L1802
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Remove this unused method parameter "seq". Why is this an issue? 2 years ago L1874
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / io/.../main/java/org/red5/io/utils/TlsUtils.java

Remove this unused method parameter "i". Why is this an issue? last year L141
Code Smell Major Open Not assigned 5min effort Comment cert, unused

Red5 / server/.../java/org/red5/server/net/rtmp/InboundHandshake.java

Remove these unused method parameters. Why is this an issue? 4 years ago L425
Code Smell Major Open Not assigned 5min effort Comment cert, unused

8 of 8 shown

Ici c'est un peu plus particulier : il s'agit de méthodes qui sont appelées mais qui n'utilisent pas tous leurs paramètres.
L'analyse est plus complexe. On privilégie l'hypothèse selon laquelle le développeur ne s'en est pas rendu compte.

Cependant, cette interprétation doit être nuancée, car ce phénomène apparaît plusieurs fois à des endroits différents du projet, parfois avec le même paramètre. Il est donc possible qu'il existe une raison liée à la conception du système.

Pour l'instant, cette question reste ouverte et sera analysée plus en détail lors de la deuxième phase du projet.

6. Synthèse critique

L'étude de Red5-Server met en évidence un projet **solide, ancien et techniquement ambitieux**, dont la structure générale reste sérieuse malgré l'accumulation naturelle de dette technique.

Points forts :

- projet toujours maintenu ;
- architecture globalement modulaire ;
- documentation présente et exploitable ;
- noms métier souvent cohérents ;
- usage structuré des pull requests ;
- forte spécialisation technique cohérente avec le domaine du streaming.

Points de vigilance :

- documentation dispersée entre plusieurs fichiers ;
- couverture de test peu lisible et outillage à fiabiliser ;
- présence de classes très volumineuses ;
- complexité cyclomatique élevée dans certaines zones ;
- problèmes localisés de nommage, de shadowing et de paramètres inutilisés ;
- nombreux nombres magiques à expliciter davantage.

RED5